# Cahier des charges
# Application de gestion d’événements avec inscription en ligne

*Version de cadrage fonctionnel et technique - architecture microservices inspirée des principes SOA*

## Sujet 2 – **Application de gestion d’événements avec inscription**

Cette plateforme permet à des organisateurs d'événements de **publier des événements** (gratuits ou payants), et aux utilisateurs de **s’inscrire en ligne**. Le but est de faciliter la communication autour des événements et la gestion des participants.

**Utilisateurs :**

- Participant
- Organisateur
- Admin

**Fonctionnalités :**

- Inscription, connexion
- Création d’événements (titre, lieu, date, description, image, capacité, public/privé)
- Système d’inscription avec liste d’attente
- Billet électronique généré (QR code facultatif)
- Espace personnel :
  - Mes événements créés
  - Mes participations
- Email/SMS de rappel (simulation)
- Interface admin pour modération
- Page publique avec calendrier filtrable par thème/lie

| | |
|---|---|
| **Objet** | Plateforme web permettant aux organisateurs de publier des événements et aux participants de s’inscrire en ligne. |
| **Acteurs** | Participant - Organisateur - Administrateur |
| **Orientation architecturale** | Microservices, contrats d’API standardisés, orchestration des flux métier, communication synchrone et asynchrone. |
| **Périmètre couvert** | Inscription, authentification, gestion des événements, liste d’attente, billet électronique, notifications, administration, calendrier public. |

# 1. Contexte et vision

L'application cible doit centraliser la publication d'événements, la gestion des inscriptions et la communication avec les participants. Elle doit supporter des événements gratuits ou payants, publics ou privés, avec capacité limitée, liste d'attente et génération de billets électroniques. Le besoin principal est de disposer d'une plateforme évolutive, modulaire et réutilisable, capable d'intégrer de nouveaux services sans remettre en cause l'ensemble du système.

## 1.1 Contexte

Le produit s'adresse à trois profils d'utilisateurs : participants, organisateurs et administrateurs. La plateforme doit offrir une expérience web claire pour la consultation et l'inscription, tout en fournissant un back-office complet pour la création, la modération et le pilotage des événements.

## 1.2 Objectifs

- Permettre la publication et la diffusion d'événements de manière simple et sécurisée.
- Automatiser le cycle d'inscription : demande, validation, gestion de la capacité, liste d'attente et confirmation.
- Mettre à disposition un billet électronique pouvant inclure un QR code.
- Informer les participants par e-mail et, en simulation dans un premier temps, par SMS de rappel.
- Fournir à l'administration des outils de modération, de supervision et de reporting.
- Adopter une architecture microservices cohérente avec les principes SOA pour limiter le couplage et favoriser l'interopérabilité.

# 2. Périmètre fonctionnel

## 2.1 Acteurs

| Acteur | Rôle principal | Fonctions attendues |
|---|---|---|
| Participant | Consommateur d'événements | Créer un compte, consulter le calendrier, filtrer, s'inscrire, payer si nécessaire, télécharger son billet, suivre ses participations, recevoir des rappels. |
| Organisateur | Créateur et gestionnaire d'événements | Créer, modifier, publier ou archiver des événements, gérer les capacités, consulter les inscrits, valider certaines demandes, exporter les listes. |
| Administrateur | Supervision et gouvernance | Modérer les contenus, gérer les utilisateurs et rôles, surveiller la plateforme, traiter les incidents, |

| | | |
|---|---|---|
| | | auditer les actions et produire des statistiques. |

## 2.2 Fonctionnalités détaillées

### Gestion des comptes et accès

- Inscription et connexion des utilisateurs.
- Réinitialisation de mot de passe.
- Gestion des rôles : participant, organisateur, administrateur.
- Option d'activation/désactivation des comptes.

### Gestion des événements

- Création d'un événement avec titre, description, thème, image, lieu, date et heure, capacité, nature publique/privée et tarification.
- Publication différée ou immédiate.
- Gestion des statuts : brouillon, publié, complet, clos, archivé, annulé.
- Possibilité de joindre des conditions ou consignes particulières.

### Inscriptions et capacité

- Inscription directe si des places sont disponibles.
- Basculer automatiquement en liste d'attente lorsque la capacité maximale est atteinte.
- Gestion des désistements et promotion automatique d'un participant depuis la liste d'attente.
- Contrôle des quotas et de l'éligibilité éventuelle.

### Billetterie et participation

- Génération d'un billet électronique au format PDF ou image.
- QR code facultatif pour pointage ou contrôle d'accès.
- Historique des participations par utilisateur.
- Export de la liste des inscrits par événement.

### Notifications

- Confirmation d'inscription, de mise en liste d'attente, de promotion et de rappel d'événement.
- Notifications par e-mail et simulation SMS dans la première version.
- Journalisation des envois et des échecs.

### Administration et modération

- Tableau de bord d'administration.
- Recherche multi-critères sur les événements et les utilisateurs.
- Modération des événements signalés ou en attente de validation.
- Consultation des logs d'activité et des indicateurs clés.

### Portail public

- Page d’accueil avec calendrier filtrable par thème, date, lieu et type d’événement.
- Fiches détaillées des événements publics.
- Recherche textuelle et filtres dynamiques.

## 3. Exigences non fonctionnelles

| Exigence | Description |
|---|---|
| Performance | Temps de réponse cible inférieur à 2 secondes pour les opérations courantes hors pics exceptionnels. |
| Disponibilité | La plateforme doit rester disponible pendant les plages de consultation et d’inscription avec mécanismes de reprise sur incident. |
| Scalabilité | Possibilité d’augmenter indépendamment la capacité des services les plus sollicités (catalogue, inscription, notification). |
| Sécurité | Chiffrement TLS, contrôle d’accès par rôles, journalisation des actions sensibles, protection contre les abus d’API. |
| Maintenabilité | Séparation claire des responsabilités, documentation des contrats, tests automatisés, observabilité centralisée. |
| Interopérabilité | APIs standardisées, formats d’échange homogènes et événements métiers publiés sur le bus pour les intégrations futures. |
| Traçabilité | Audit des créations, modifications, annulations, validations et envois de notifications. |
| Portabilité | Déploiement conteneurisé compatible avec environnement cloud ou on-premise. |

## 4. Architecture cible

L'architecture retenue est une architecture microservices alignée sur les principes d'une architecture orientée services : contrats standardisés, faible couplage, communication par messages, découvrabilité, composition/orchestration et réutilisabilité des services. Chaque microservice doit rester autonome, centré sur un domaine métier cohérent et déployable indépendamment.

### 4.1 Principes d'architecture

- Chaque service expose un contrat d'API explicite (REST/JSON, éventuellement événements asynchrones).
- Les services doivent être stateless autant que possible ; l'état métier persistant est externalisé dans leurs bases de données.
- La logique de processus transverse est portée par des orchestrations et non par un couplage point-à-point incontrôlé.
- Les services techniques transverses (logs, sécurité, supervision, configuration) sont mutualisés.
- Les échanges synchrones sont réservés aux consultations et validations immédiates ; les échanges asynchrones sont privilégiés pour notifications, génération de billets et mises à jour dérivées.

### 4.2 Vue macro

La solution se structure en cinq zones : (1) couche de présentation web/mobile, (2) API Gateway, (3) couche des microservices métier, (4) couche d'intégration et de communication asynchrone, (5) couche de données et de stockage. Des services de QoS transverses complètent l'ensemble : sécurité, monitoring, logging, configuration et gestion des erreurs.

### 4.3 Microservices proposés

| Microservice | Responsabilités | Données principales | Mode d'échange |
|---|---|---|---|
| Identity & Access | Authentification, autorisation, rôles, sessions, tokens | utilisateurs, rôles, sessions | REST |
| User Profile | Profil utilisateur, préférences, historique d'activité | profils, préférences | REST |
| Event Management | Création, édition, publication et cycle de vie des événements | événements, thèmes, lieux, images | REST |
| Catalog & Search | Exposition publique, recherche, filtres, calendrier | index de recherche, vues catalogue | REST |
| Registration | Inscriptions, capacité, | inscriptions, quotas, | REST + événements |

| | | | |
|---|---|---|---|
| | liste d'attente, annulation | statuts | |
| Ticketing | Génération des billets et QR codes | billets, artefacts PDF/PNG | événements asynchrones |
| Notification | Envois e-mail/SMS et templates | modèles, journaux d'envoi | événements asynchrones |
| Admin & Moderation | Validation, modération, tableaux de bord, audit | signalements, décisions, audits | REST |
| Payment (optionnel) | Paiement et rapprochement pour événements payants | transactions, statuts de paiement | REST + webhooks |
| Media | Stockage et distribution des images et pièces associées | fichiers, métadonnées | REST |

### 4.4 Composants d'infrastructure

- API Gateway : point d'entrée unique, routage, limitation de débit, authentification centralisée et exposition des versions d'API.
- Service discovery / registre de services : référencement des services déployés et de leurs points d'accès.
- Bus de messages : diffusion des événements métiers (inscription créée, promotion depuis la liste d'attente, billet généré, rappel envoyé).
- Configuration centralisée : variables et paramètres de service par environnement.
- Observabilité : logs centralisés, métriques, traces distribuées et alerting.
- Stockage objet : conservation des images et billets générés.

### 4.5 Positionnement par rapport à l'ESB

Conformément aux recommandations du cours SOA, un ESB peut être utile pour la transformation, le routage et l'interopérabilité, mais il n'est pas obligatoire. Pour ce projet, l'option recommandée consiste à retenir un modèle plus léger centré sur une API Gateway et un bus de messages. L'usage d'un ESB ne sera envisagé que si le système doit intégrer plusieurs applications hétérogènes, protocoles propriétaires ou transformations complexes de formats.

## 5. Scénarios métier clés

### 5.1 Inscription standard à un événement

**1.** Le participant consulte le catalogue public ou son espace connecté.

**2.** Il sélectionne un événement et soumet sa demande d'inscription.

**3.** Le service Registration vérifie la capacité et les règles d'éligibilité.

**4.** Si la place est disponible, l'inscription est confirmée et un événement métier est publié.

**5.** Le service Ticketing génère le billet, puis Notification envoie la confirmation.

### 5.2 Gestion de liste d'attente

**1.** Si l'événement est complet, la demande est enregistrée en liste d'attente.

**2.** En cas d'annulation, Registration promeut automatiquement le premier candidat éligible.

**3.** Ticketing et Notification sont déclenchés après promotion effective.

### 5.3 Publication d'un événement par l'organisateur

**1.** L'organisateur crée un brouillon avec les métadonnées et l'image associée.

**2.** Le service Event Management valide la cohérence des informations.

**3.** Selon la politique définie, l'événement est publié directement ou soumis à modération.

**4.** Le service Catalog & Search indexe l'événement publié pour l'affichage public.

## 6. Modèle de données de haut niveau

- Utilisateur : identité, coordonnées, rôle, état du compte.
- Événement : titre, description, catégorie, lieu, date, capacité, visibilité, tarification, statut.
- Inscription : participant, événement, statut (confirmée, attente, annulée, refusée), horodatage.
- Billet : référence, code ou QR, état d'émission, fichier associé.
- Notification : canal, contenu, date d'envoi, statut technique.
- Audit : acteur, action, cible, résultat, horodatage.

## 7. Sécurité et conformité

- Authentification par jetons sécurisés et gestion des sessions.
- Autorisation fine par rôle et par ressource.
- Chiffrement des flux HTTPS/TLS.
- Journalisation des accès, des changements de statut et des actions d'administration.
- Protection contre l'abus d'API : rate limiting, validation stricte des entrées, anti-bruteforce.
- Sauvegarde régulière des données critiques et plan de restauration.

## 8. Exigences d'intégration et d'exploitation

- Déploiement conteneurisé par service (Docker/Kubernetes ou alternative adaptée).
- Pipeline CI/CD avec tests unitaires, tests d'intégration et contrôle de qualité des contrats.
- Versionnement des APIs et politique de rétrocompatibilité.
- Environnements distincts : développement, intégration, préproduction, production.

- Supervision continue avec tableaux de bord techniques et fonctionnels.

## 9. Livrables attendus

- Spécifications fonctionnelles détaillées.
- Cartographie des microservices et contrats d'API.
- Modèle de données logique et dictionnaire des données.
- Maquettes des interfaces principales.
- Backlog initial et découpage MVP / versions ultérieures.
- Plan de tests et critères de recette.
- Dossier d'exploitation et d'architecture.

## 10. Découpage recommandé du projet

| Phase | Objectif | Contenu | Priorité |
|---|---|---|---|
| P1 | MVP | Authentification, gestion événements, inscription, liste d'attente, portail public, espace personnel. | Très haute |
| P2 | Billetterie | Billet électronique, QR code, export des inscrits, notifications e-mail. | Haute |
| P3 | Administration avancée | Modération, audit, statistiques, tableaux de bord. | Haute |
| P4 | Monétisation et extensions | Paiement, connecteurs externes, mobile, analytique avancée. | Moyenne |

## 11. Critères de recette

- Un organisateur peut créer et publier un événement complet sans intervention technique.
- Un participant peut s'inscrire, consulter le statut de sa demande et télécharger son billet.
- La gestion de capacité et de liste d'attente fonctionne sans doublons ni dépassement de quota.
- Les notifications attendues sont déclenchées selon les événements métiers configurés.
- Les droits d'accès empêchent un utilisateur non autorisé d'exécuter une action d'administration.
- Les journaux et métriques permettent de retracer un incident de bout en bout.

## 12. Recommandations techniques finales

Le projet devra éviter une conception monolithique déguisée. Les services doivent être découpés par domaine métier, disposer de contrats clairs et communiquer par interfaces normalisées. Les dépendances directes entre services doivent rester limitées. L'orchestration des processus, la sécurité transverse, le monitoring et la découvrabilité des services doivent être pensés dès la phase de conception. Cette approche garantit une meilleure évolutivité, une maintenance plus simple et une intégration future plus maîtrisée.

## Annexe A - Correspondance entre besoins et services

| Besoin | Service principal | Services contributifs |
|---|---|---|
| Créer un événement | Event Management | Media, Admin & Moderation |
| Afficher le calendrier public | Catalog & Search | Event Management |
| Inscrire un participant | Registration | Identity & Access, Ticketing |
| Gérer les rappels | Notification | Registration, Event Management |
| Produire les tableaux de bord | Admin & Moderation | Catalog & Search, Registration, Audit |